CONTRACT WOLF

Security Assessment

LoomDesk

Verified on October 30, 2026

SUMMARY

Project	CHAIN	METHODOLOGY
LoomDesk	Robinhood Chain	Manual & Automatic Analysis

FILES	DELIVERY	TYPE
Repository	October 30, 2026	Standard Audit



17	0	1	3	8	5	17
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

0 Critical	An exposure that can affect the contract functions in several events that can risk and disrupt the contract
1 Major	An opening & exposure to manipulate the contract in an unwanted manner
3 Medium	An opening that could affect the outcome in executing the contract in a specific situation
8 Minor	An opening but doesn't have an impact on the functionality of the contract
5 Informational	An opening that consists information but will not risk or affect the contract
17 Resolved	ContractWolf's findings has been acknowledged & resolved by the project

STATUS ✔ AUDIT PASSED

TABLE OF CONTENTS | LoomDesk

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details
Inheritance Graph
Call Graph

Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | LoomDesk

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | LoomDesk

LoomDesk team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **LoomDesk**.

AUDITING APPROACH | LoomDesk

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.

2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.

3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.

4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | LoomDesk



LoomLadder lets anyone open a liquidity position in any token's Uniswap v3 or v4 pool on Robinhood Chain as one deposit of up to 40 narrow ranges.

Token Name	Symbol	Decimal	Total Supply	Chain
Loom	LOOM	18	999,995,469.70	Robinhood Chain

SOURCE

Source	*GitHub Repository (major fixes : f0428a7, pilot relay changes : 5be4600)*
LoomLadder	*0x792cC8791AEb4D61a894DC770d38e98ae8537aFA*
LadderWork	*0x28ec5EA337D8a5b18dCAA1fbE9b7f2A09F2d0A0e*
LadderAuto	*0xB32A5491903B46A2AF2caaaEE9b7a82cef1E1c5d*
LadderKeep	*0x4188bFFaDa90E7657594bEad21835DEBBA17573e*
LadderMath	*0x826CD9B2a3b5F7b0DdfA89B861BA999Ade36AC77*
LoomZap	*0x6e292580F888f13B70335985482a2bcb526d1ffF*
PilotGas	*0x3A56f62ebba59d6531d9c24fe451B4e506D2eCDE*
LoomLock	*0xb14feF853F7a00fCffBB93F2e70831B04E8965A4*
RamsesNpmAdapter	*0xB10A6555F2bbA7Bb3D24c84BAdEEfb9D999b67b2*
SlipstreamNpmAdapter	*0x8F0b774099911826224F987D6464B02655d39518*

FINDINGS | LoomDesk



17	0	1	3	8	5	17
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

This report has been prepared to state the issues and vulnerabilities for LoomDesk through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
PILOT-SPOT-PRICE	Autopilot limits are measured at a spot price the pilot can move	LadderAuto.sol, L: 252, 268, 272	Major	Resolved
SWC-113	A payee that refuses a cut blocks every exit	LadderWork.sol, L: 568	Medium	Resolved
SWC-114	LoomZap builds ladders with no bound on the pool price	LoomZap.sol, L: 429, 437, 263	Medium	Resolved
LOCK-COMPOUND	Sealing leaves compounding on, and its leftovers are stuck	LoomLock.sol, L: 79	Medium	Resolved
CUT-OVERCHARGE	An in-kind cut keeps the 5% sale allowance	LadderWork.sol, L: 427, 436	Minor	Resolved
DISCOUNT-FLASH	Holder discount can be borrowed for one transaction	LadderKeep.sol, L: 62-63	Minor	Resolved
SELF-REFERRAL	Opening fee can be routed back to the opener's own wallets	LoomLadder.sol, L: 740; LadderAuto.sol, L: 418	Minor	Resolved
ADAPTER-ROUTE	Ramses and UP ladders sell through Uniswap's router	LadderWork.sol, L: 496; LoomZap.sol, L: 447	Minor	Resolved
TANK-SHARED	PilotGas lets any allowed ladder name whose tank pays	PilotGas.sol, L: 113-120	Minor	Resolved
LOCK-STUCK	Payouts and NFTs that reach LoomLock can never leave	LoomLock.sol, L: 28	Minor	Resolved

LOCK-HOOKDATA	Locked ladders always collect with empty hookData	LoomLock.sol, L: 87	Minor	Resolved
LOCK-STALE-PREPARE	A previous owner's preparation survives a give	LoomLock.sol, L: 63-64, 73	Minor	Resolved
SWC-134	Hard-coded 60,000 gas on soft ETH sends	LadderWork.sol, L: 565, 574	Informational	Resolved
SWC-128	Anyone can lengthen a beneficiary's list	LoomLock.sol, L: 97, 111-119	Informational	Resolved
CW-003	Adapter functions are payable but never use or return ETH	RamsesNpmAdapter.sol, L: 90-128; SlipstreamNpmAdapter.sol, L: 96-134	Informational	Resolved
SWC-131	Unused declarations and a statement with no effect	LoomLadder.sol, L: 8, 15; LadderMath.sol, L: 14; LadderAuto.sol, L: 491	Informational	Resolved
SWC-103	Floating pragmas across three versions	LoomLadder.sol, L: 2; LadderKeep.sol, L: 2; RamsesNpmAdapter.sol, L: 2	Informational	Resolved

SWC ATTACKS | LoomDesk

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	Passed
SWC-101	Integer Overflow and Underflow	Passed
SWC-102	Outdated Compiler Version	Passed
SWC-103	Floating Pragma	Passed
SWC-104	Unchecked Call Return Value	Passed
SWC-105	Unprotected Ether Withdrawal	Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	Passed
SWC-107	Reentrancy	Passed
SWC-108	State Variable Default Visibility	Passed
SWC-109	Uninitialized Storage Pointer	Passed
SWC-110	Assert Violation	Passed
SWC-111	Use of Deprecated Solidity Functions	Passed
SWC-112	Delegate call to Untrusted Callee	Passed
SWC-113	DoS with Failed Call	Passed
SWC-114	Transaction Order Dependence	Passed
SWC-115	Authorization through tx.origin	Passed
SWC-116	Block values as a proxy for time	Passed
SWC-117	Signature Malleability	Passed
SWC-118	Incorrect Constructor Name	Passed
SWC-119	Shadowing State Variables	Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	Passed
SWC-121	Missing Protection against Signature Replay Attacks	Passed
SWC-122	Lack of Proper Signature Verification	Passed

ID	Description	Status
SWC-123	Requirement Violation	Passed
SWC-124	Write to Arbitrary Storage Location	Passed
SWC-125	Incorrect Inheritance Order	Passed
SWC-126	Insufficient Gas Griefing	Passed
SWC-127	Arbitrary Jump with Function Type Variable	Passed
SWC-128	DoS With Block Gas Limit	Passed
SWC-129	Typographical Error	Passed
SWC-130	Right-To-Left-Override control character(U+202E)	Passed
SWC-131	Presence of unused variables	Passed
SWC-132	Unexpected Ether balance	Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	Passed
SWC-134	Message call with hardcoded gas amount	Passed
SWC-135	Code With No Effects	Passed
SWC-136	Unencrypted Private Data On-Chain	Passed

CW ASSESSMENT | LoomDesk

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✔
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✔
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✔
CW-004	Custom Modifier	major recheck for custom modifier logic	✔
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✔
CW-006	Multiple Calls	Functions with multiple internal calls	✔
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✔
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✔
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✔
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✔
CW-011	Commented Code	Removal of commented/unused code lines	✔
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✔

FIXES & RECOMMENDATION

PILOT-SPOT-PRICE | Autopilot limits are measured at a spot price the pilot can move

`_reshapeDue`, `_exitDue`, the `MAX_RESHAPE_LOSS_BPS` check and `lossOk` all read the pool's current `sqrtP`, and the pilot picks the `price` bounds and `mins` itself. A pilot that moves the price first can trigger a re-lay or a stop loss, lay every rung around the false price, then trade back through them. The loss checks only see the `_fit` swap, so a leaked keeper key, or any address named with `setPilot`, can take far more than 20% a day.

```
// LadderAuto.sol - L252: due when the spot tick has left the rungs
if (job.piloted && !entering)
    _reshapeDue(a, l.nft, l.tokenIds, c.stateView, V4Pool.id(key));

// L265-L268: before and after are both valued at the same moved c.sqrtP
uint256 before = _worth(held, c.sqrtP);
_fit(c, s, key, rungs, held, hookData);
if (_worth(held, c.sqrtP) < (before * (BPS - MAX_RESHAPE_LOSS_BPS)) / BPS)
    revert TooCostly();

// LoomLadder.sol - L622: the bounds come from the pilot itself
_checkPrice(_sqrtPriceOf(l, e), price[0], price[1]);
```

Recommendation

Before any pilot action, compare the spot price with a reference the pilot cannot move in one block (a v3 `observe()` TWAP or an oracle) and refuse the action when they differ by more than a small band. Value `lossOk` at that reference, and put `setPilot` behind a timelock.

```
// LoomLadder.sol - autoRun, for the pilot only
if (job.piloted) {
    uint160 spot = _sqrtPriceOf(l, e);
    uint160 ref = _twapSqrtPrice(l, e, 30 minutes); // v3 observe(), or an oracle
    uint256 gap = spot > ref ? spot - ref : ref - spot;
    if (gap * BPS > uint256(ref) * PILOT_MAX_DRIFT_BPS) revert PriceOutOfBounds();
}

// LadderAuto._reshape: the day's loss counted at the reference, not at spot
// heldBefore: a copy of `held` taken before _fit
uint256 lost = _valueIn(heldBefore, ref, c.q0) - _valueIn(held, ref, c.q0);
if (!lossOk(pilotLoss, job.ladderId, e.value + fees, lost, block.timestamp))
    revert TooCostly();
```

SWC-113 | A payee that refuses a cut blocks every exit

`_payCut` pays the developer and referrers softly, but the book's share goes to desk or sweeper through `_send(..., false)`, which reverts on failure. `collect`, `close`, `closePart`, `withdraw` and `autoRun` all take a cut once any fee has accrued. If USDG freezes the desk, a restricted stock token refuses the sweeper, or `setPayees` names a reverting address, users cannot exit until the owner runs `returnLadder`.

```
// LadderWork.sol - L563-L569
function _payCut(Ctx memory c, address currency, uint256 amount, address to) private {
    if (amount == 0) return;
    if (to != address(0) && _send(currency, to, amount, true)) return;
    uint256 toDev = c.dev == address(0) ? 0 : (amount * c.devShareBps) / BPS;
    if (toDev > 0 && !_send(currency, c.dev, toDev, true)) toDev = 0;
    // a refusal here reverts the user's close, collect or withdraw
    _send(currency, currency == c.usdg ? c.desk : c.sweeper, amount - toDev, false);
}
```

Recommendation

Pay the book softly as well, and credit whatever it refuses to `owed[payee][currency]` the way `payOrOwe` already does for users. A frozen or reverting payee then only delays the protocol's revenue, never a user's exit.

```
// LadderWork._payCut: the book paid softly, a refusal kept as owed
address payee = currency == c.usdg ? c.desk : c.sweeper;
uint256 rest = amount - toDev;
if (!_send(currency, payee, rest, true)) {
    owed[payee][currency] += rest; // the ladder's mappings, handed in
    totalOwed[currency] += rest;
    emit Owed(payee, currency, rest);
}
```

SWC-114 | LoomZap builds ladders with no bound on the pool price

`_open4` and `_open3` call the ladder with `sqrtPriceMin = 0` and `sqrtPriceMax = type(uint160).max`, and `_amounts` funds each rung from wherever the price stands after the legs. A transaction placed before `zapBuild`, or before a zap with `swapAmount = 0`, can push the pool so buy rungs land above the fair price, then sell into them. `_feeUsdg` also buys an exact USDG output with no cap on the WETH spent.

```
// LoomZap.sol - L429 (L437 is the same for v3)
ladderId = ladder.openV4For{value: value}(
    msg.sender, key, built, 0, type(uint160).max, deadline, hookData, credit
);

// LoomZap.sol - L263: exact output, no price limit and no maximum input
IV3PoolZ(pool).swap(address(this), zeroForOne, -int256(fee - have),
    zeroForOne ? MIN_SQRT : MAX_SQRT, "");
```

Recommendation

Take `sqrtPriceMin` and `sqrtPriceMax` from the caller in every zap entry point and pass them to `openV4For` and `openV3For`, which already enforce them. In `_feeUsdg`, refuse the purchase when it spends more WETH than a caller-supplied maximum.

```
function zapBuild(..., uint160[2] calldata price, uint256 maxFeeWeth)
    external payable nonReentrant returns (uint256 ladderId)
{
    ...
    ladderId = ladder.openV4For{value: value}(
        msg.sender, key, built, price[0], price[1], deadline, hookData, credit
    );
}

// _feeUsdg: the exact-output buy may not cost more than the caller allowed
uint256 wethBefore = IERC20(weth).balanceOf(address(this));
IV3PoolZ(pool).swap(address(this), zeroForOne, -int256(fee - have), limit, "");
uint256 spent = wethBefore - IERC20(weth).balanceOf(address(this));
if (spent > maxFeeWeth) revert TooLittleOut();
```

LOCK-COMPOUND | Sealing leaves compounding on, and its leftovers are stuck

`seal` switches the autopilot off but not compounding, although LoomLock, as the ladder's owner, may call `setCompound(ladderId, false)`. A ladder locked with compounding on is compounded by the sweeper or the pilot whenever they choose, so the beneficiary's fees go into the rungs instead. What each compound cannot add is paid to the ladder's owner, LoomLock, which has no way to move tokens or claim owed ETH.

```
// LoomLock.sol - L79: the autopilot is switched off, compounding is not
if (ladder.autoOf(ladderId) != 0) ladder.setAuto(ladderId, 0);

// LoomLadder.sol - L490-L497: the sweeper or pilot may still compound,
// and the leftovers go to l.owner, which is now LoomLock
if (l.owner != msg.sender
    && !(compounds[ladderId] && (msg.sender == sweeper || msg.sender == pilot)))
    revert NotYours();
...
_payOrOwe(l.currency0, l.owner, over[0]);
_payOrOwe(l.currency1, l.owner, over[1]);
```

Recommendation

Switch compounding off in `seal` the same way the autopilot is switched off, and correct the comment that says the lock cannot do it. The setting is the owner's, and LoomLock is the owner by then.

```
interface ILadderLocked {
    ...
    function compounds(uint256 ladderId) external view returns (bool);
    function setCompound(uint256 ladderId, bool on) external;
}

// seal
if (ladder.autoOf(ladderId) != 0) ladder.setAuto(ladderId, 0);
if (ladder.compounds(ladderId)) ladder.setCompound(ladderId, false);
```

CUT-OVERCHARGE | An in-kind cut keeps the 5% sale allowance

When the money side is short, `cut` sizes the token slice at its spot value plus need / 20, to cover the pool fee and price push of a sale. If `_sellInPool` returns zero (no floor, a hook tax, the price limit, a failed router call), the whole need is paid in kind although nothing was sold. The user then pays the short part of every such cut 5% over its value.

```
// LadderWork.sol - L426-L438
uint256 need = _atSpot(c.sqrtP, c.q0, short);
if (tok != KEPT) need += need / 20;
if (need > amt[t]) need = amt[t];
uint256 got = tok == KEPT ? 0 : _sellInPool(c, s, tok, need, hookData);
amt[t] -= need;
if (got > 0) {
    amt[m] += got;
    take += short < got ? short : got;
} else {
    _payCut(c, tok, need, to); // no sale was made, yet the 5% stays in
    emit Cut(ladderId, kind, tok, need);
}
```

Recommendation

Keep the allowance only for a sale that happens. On the in-kind path pay the slice at its spot value and leave the allowance in the user's amounts.

```
uint256 atSpot = _atSpot(c.sqrtP, c.q0, short);
uint256 need = tok == KEPT ? atSpot : atSpot + atSpot / 20;
if (need > amt[t]) need = amt[t];
uint256 got = tok == KEPT ? 0 : _sellInPool(c, s, tok, need, hookData);
if (got > 0) {
    amt[t] -= need;
    amt[m] += got;
    take += short < got ? short : got;
} else {
    uint256 slice = atSpot < amt[t] ? atSpot : amt[t]; // no sale, no allowance
    amt[t] -= slice;
    _payCut(c, tok, slice, to);
    emit Cut(ladderId, kind, tok, slice);
}
```

DISCOUNT-FLASH | Holder discount can be borrowed for one transaction

`feeBpsFor` grants `discountBps` (30% live) to any owner whose `discountToken` balance reaches `discountMinBalance` (100,000 LOOM live) at the moment of the call. An owner contract can flash borrow, or buy and resell, the LOOM around `collect` or `withdraw`, and one balance can be passed between wallets for any number of ladders. `withdraw` charges `feeBps` of the whole ladder, so there the saving is 1.5% of the position.

```
// LadderKeep.sol - L59-L64
bps = feeBps;
if (discountToken == address(0) || discountBps == 0 || bps == 0) return bps;
(bool ok, bytes memory ret) =
    discountToken.staticcall(abi.encodeCall(IERC20.balanceOf, (owner_)));
// the balance at this instant: borrowed tokens count as held
if (ok && ret.length >= 32 && abi.decode(ret, (uint256)) >= discountMinBalance)
    bps -= (bps * discountBps) / 10_000;
```

Recommendation

Base the discount on tokens held over time, such as a stake locked for a minimum period, so that LOOM held only inside one transaction earns nothing.

```
struct Stake { uint128 amount; uint64 since; }
mapping(address => Stake) public stakeOf; // set by a staking contract
uint256 public constant MIN_HOLD = 7 days;

function _discounted(address owner_) internal view returns (bool) {
    Stake memory st = stakeOf[owner_];
    return st.amount >= discountMinBalance && block.timestamp >= st.since + MIN_HOLD;
}
```

SELF-REFERRAL | Opening fee can be routed back to the opener's own wallets

`_openFee` accepts any `credit.referrer` other than the owner, and `chargeOpenFee` pays the copy share to the owner of any other open ladder in the same pair. An opener who names a second wallet as referrer and copies a tiny ladder held by that wallet gets `refBps + copyBps` back, 20 of the 25 bps live, so the book keeps a fifth of the opening fee.

```
// LoomLadder.sol - L740
if (referrerOf[owner_] == address(0) && c.referrer != address(0)
    && c.referrer != owner_ && (msg.sender == owner_ || msg.sender == zap)) {
    referrerOf[owner_] = c.referrer;
}

// LadderAuto.sol - L418
if (src.owner != address(0) && src.owner != owner_
    && src.currency0 == l.currency0 && src.currency1 == l.currency1)
    sp.author = src.owner;
```

Recommendation

Pay referral shares only to referrers the operator has registered, and pay the copy share only for a copied ladder that has been open for a minimum time and holds a minimum value, so a throwaway second wallet cannot collect either.

```
mapping(address => bool) public referrerApproved; // owner-curated
uint256 public constant MIN_COPY_AGE = 7 days;

// _openFee
if (referrerOf[owner_] == address(0) && referrerApproved[c.referrer]
    && c.referrer != owner_ && (msg.sender == owner_ || msg.sender == zap)) {
    referrerOf[owner_] = c.referrer;
}

// chargeOpenFee: the copied ladder must be established
if (src.owner != address(0) && src.owner != owner_ && samePair
    && block.timestamp >= openedAt[copyOf] + MIN_COPY_AGE) sp.author = src.owner;
```

ADAPTER-ROUTE | Ramses and UP ladders sell through Uniswap's router

For a v3 ladder `_sellInPool` always calls Uniswap's `SwapRouter02` with the ladder pool's `fee()`, and `zapClose` looks the pool up in Uniswap's factory. For ladders opened through `RamsesNpmAdapter` or `SlipstreamNpmAdapter` both resolve to a different Uniswap pool, which usually does not exist, so every token-side cut falls back to in kind and `zapClose` reverts or trades in the wrong pool.

```
// LadderWork.sol - L496: SwapRouter02 derives a Uniswap pool from this fee
return _routerSwap(c.router, tokenIn, inIs0 ? p.key.currency1 : p.key.currency0,
    IV3PoolW(p.v3Pool).fee(), amountIn, minOut);

// LoomZap.sol - L447: Uniswap's factory, with l.fee holding the tick spacing
address pool = IV3FactoryZ(INpmZ(npm).factory())
    .getPool(l.currency0, l.currency1, l.fee);
```

Recommendation

Swap in the ladder's own pool directly, calling swap on `p.v3Pool` with a callback that checks `msg.sender` against the pool being swapped, as `LoomZap` already does. In `zapClose`, take the pool from `entryOf(ladderId).pool` instead of Uniswap's factory.

```
// LadderWork: a direct swap in the ladder's own pool, any v3 fork
s.v3Pool = p.v3Pool; // the only pool the swap callback will pay
try IV3PoolW(p.v3Pool).swap(address(this), inIs0, int256(amountIn), limit,
    abi.encode(tokenIn, minOut)) returns (int256 d0, int256 d1) {
    got = uint256(-(inIs0 ? d1 : d0)); // the callback reverts below minOut
} catch {}
s.v3Pool = address(0);

// LoomZap.zapClose
(, bytes32 poolId,) = ladder.entryOf(ladderId);
address pool = address(uint160(uint256(poolId)));
```

TANK-SHARED | PilotGas lets any allowed ladder name whose tank pays

`run` takes the owner to charge from `ILadderPilot(ladder).ladder(ladderId).owner` and bills all gas the `autoRun` call burns, with no cap per run. Tanks are keyed by wallet across every contract `setLadder` allows, so a contract allowed by mistake, or by a leaked owner key, can report any depositor as the owner and burn gas until that tank is empty, paid to the operator.

```
// PilotGas.sol - L112-L120
if (msg.sender != operator || !ladders[ladder]) revert NotAllowed();
address owner_ = ILadderPilot(ladder).ladder(ladderId).owner; // the ladder's word
if (balanceOf[owner_] == 0) revert NoGas();
(usdgOut, amount0, amount1) = ILadderPilot(ladder)
    .autoRun(kind, ladderId, asUsdg, mins, price, deadline, hookData);
uint256 used = start - gasleft() + overheadGas; // no upper bound
uint256 at = tx.gasprice < maxGasPrice ? tx.gasprice : maxGasPrice;
uint256 charge = used * at;
```

Recommendation

Key tanks by ladder contract and owner, so a deposit can only pay for the ladder it was made for, and cap the gas charged per run so one call can never take more than a normal action costs.

```
mapping(address => mapping(address => uint256)) public balanceOf; // ladder => owner
uint256 public constant MAX_RUN_GAS = 3_000_000;

uint256 used = start - gasleft() + overheadGas;
if (used > MAX_RUN_GAS) used = MAX_RUN_GAS;
uint256 charge = used * (tx.gasprice < maxGasPrice ? tx.gasprice : maxGasPrice);
if (charge > balanceOf[ladder][owner_]) revert NoGas();
balanceOf[ladder][owner_] -= charge;
```

LOCK-STUCK | Payouts and NFTs that reach LoomLock can never leave

LoomLock owns every ladder it holds, so the ladder pays it whatever goes to an owner: autopilot exits, harvests and reshape leftovers run between `give` and `seal`, and the position NFTs when the ladder's owner calls `returnLadder`. LoomLock has no `receive`, no token or NFT transfer and no `claimOwed` call, so all of it stays there for good, and a returned ladder's fees can never be collected again.

```
// LoomLadder.sol - L630-L632: autoRun pays the ladder's owner, LoomLock before seal
_payOrOwe(usdg, owner_, usdgOut);
_payOrOwe(c0, owner_, amount0);
_payOrOwe(c1, owner_, amount1);

// LadderKeep.sol - L92-L94: returnLadder sends the NFTs to the owner, LoomLock
address to = l.owner;
for (uint256 i; i < l.tokenIds.length; ++i)
    IERC721K(l.nft).transferFrom(address(this), to, l.tokenIds[i]);
```

Recommendation

Refuse to `prepare` a ladder whose autopilot or compounding is on, so nothing can pay the lock before it is sealed, and have `returnLadder` refuse ladders owned by the lock, whose NFTs it could only strand there.

```
// LoomLock.prepare: nothing may pay the lock between the give and the seal
if (ladder.autoOf(ladderId) != 0 || ladder.compounds(ladderId)) revert NotPrepared();

// LoomLadder.returnLadder: a locked ladder has nowhere to go
if (_ladders[ladderId].owner == lock) revert NotAllowed();
```

LOCK-HOOKDATA | Locked ladders always collect with empty hookData

LoomLadder lets every v4 call carry the caller's `hookData`, but `LoomLock.collect` always passes an empty one. In a pool whose hook needs data to remove liquidity or to pay out fees, the zero-liquidity decrease behind each collect reverts, so the beneficiary can never receive that locked ladder's fees. Nothing else may collect them, and the ladder can never be closed.

```
// LoomLock.sol - L84-L89
function collect(uint256 ladderId, address to)
    external nonReentrant returns (uint256 amount0, uint256 amount1)
{
    if (beneficiary[ladderId] != msg.sender) revert NotTheBeneficiary();
    if (to == address(0)) revert ZeroAddress();
    (amount0, amount1) = ladder.collect(ladderId, to, "");
    emit Collected(ladderId, to, amount0, amount1);
}
```

Recommendation

Let the beneficiary pass `hookData` through, as every LoomLadder entry point does.

```
function collect(uint256 ladderId, address to, bytes calldata hookData)
    external nonReentrant returns (uint256 amount0, uint256 amount1)
{
    if (beneficiary[ladderId] != msg.sender) revert NotTheBeneficiary();
    if (to == address(0)) revert ZeroAddress();
    (amount0, amount1) = ladder.collect(ladderId, to, hookData);
    emit Collected(ladderId, to, amount0, amount1);
}
```

LOCK-STALE-PREPARE | A previous owner's preparation survives a give

`prepare` stores a beneficiary per ladder and `preparedBy` records who named it, but nothing clears the entry when the ladder later changes hands, and `seal` never reads `preparedBy`. If Alice prepares a ladder, gives it to Bob, and Bob locks it without preparing again, `seal` makes Alice's choice the beneficiary of Bob's ladder. A site check that only asks whether a ladder is prepared would let that give through.

```
// LoomLock.sol - L63-L64 and L73-L75
prepared[ladderId] = beneficiary_;
preparedBy[ladderId] = msg.sender; // written, never checked
...
address b = prepared[ladderId]; // may be a previous owner's choice
if (b == address(0)) revert NotPrepared();
beneficiary[ladderId] = b;
```

Recommendation

Give a preparation a short expiry that `seal` checks, and add a view that reports a ladder as prepared only while `preparedBy` is still its owner, so neither the contract nor the site can accept a previous owner's choice.

```
mapping(uint256 => uint64) public preparedAt; // set in prepare

function isPrepared(uint256 ladderId) external view returns (bool) {
    return prepared[ladderId] != address(0)
        && preparedBy[ladderId] == ladder.ladder(ladderId).owner;
}

// seal: only a recent preparation counts
if (block.timestamp > preparedAt[ladderId] + 1 hours) revert NotPrepared();
```

SWC-134 | Hard-coded 60,000 gas on soft ETH sends

Referrer, copy author and developer shares paid in ETH are sent with exactly `60_000` gas, and the share goes to the book when that call fails. A smart contract wallet whose `receive` needs more gas, such as a proxy with a fallback handler or a 4337 account, silently loses its share. Recipients on the `payOrOwe` path are safe, since a failure there is kept as owed.

```
// LadderWork.sol - L565: a refused share is handed to the book
if (to != address(0) && _send(currency, to, amount, true)) return;

// LadderWork.sol - L574
(ok,) = soft ? to.call{value: amount, gas: 60_000}("") : to.call{value: amount}("");
```

Recommendation

Credit a refused referrer, author or developer share to `owed[to][currency]` instead of handing it to the book, so the recipient can claim it with `claimOwed` whatever its wallet needs.

```
if (to != address(0)) {
    if (!_send(currency, to, amount, true)) _owe(to, currency, amount); // claimable
    return;
}
```

SWC-128 | Anyone can lengthen a beneficiary's list

`_unlist` walks `_lockedBy[who]` one entry at a time, and anyone can add entries to another wallet's list by locking dust ladders with that wallet as beneficiary, or by passing ladders to it with `setBeneficiary`. Once enough entries sit ahead of a ladder, `setBeneficiary` for it runs out of gas. `collect` does not touch the list and keeps working.

```
// LoomLock.sol - L111-L120
function _unlist(address who, uint256 ladderId) private {
    uint256[] storage list = _lockedBy[who];
    for (uint256 i = 0; i < list.length; ++i) {
        if (list[i] == ladderId) {
            list[i] = list[list.length - 1];
            list.pop();
            return;
        }
    }
}
```

Recommendation

Keep each ladder's place in its beneficiary's list, as `LadderKeep.list` and `unlist` already do for owners, so removing one costs the same whatever the list's length.

```
mapping(uint256 => uint256) private _lockedAt; // place in the list, plus one

function _list(address who, uint256 ladderId) private {
    _lockedBy[who].push(ladderId);
    _lockedAt[ladderId] = _lockedBy[who].length;
}

function _unlist(address who, uint256 ladderId) private {
    uint256[] storage list = _lockedBy[who];
    uint256 at = _lockedAt[ladderId];
    uint256 last = list[list.length - 1];
    list[at - 1] = last;
    _lockedAt[last] = at;
    list.pop();
    delete _lockedAt[ladderId];
}
```

CW-003 | Adapter functions are payable but never use or return ETH

`mint`, `increaseLiquidity`, `decreaseLiquidity`, `collect` and `burn` are `payable` in both adapters, copying Uniswap's signatures, but none forwards or refunds `msg.value`, and neither adapter can withdraw ETH. Any ETH sent with one of these calls is locked in the adapter for good. The ladder never sends value, so only third-party callers are exposed.

```
// RamsesNpmAdapter.sol - L90 (the same in SlipstreamNpmAdapter.sol, L96)
function mint(MintParams calldata p) external payable returns (
    uint256 tokenId, uint128 liquidity, uint256 amount0, uint256 amount1
)

// L106, L118, L123, L128
function increaseLiquidity(IncreaseLiquidityParams calldata p) external payable ...
function decreaseLiquidity(DecreaseLiquidityParams calldata p) external payable ...
function collect(CollectParams calldata p) external payable ...
function burn(uint256 tokenId) external payable ...
```

Recommendation

Drop `payable` from the five functions. The ladder calls them without value and the selector does not depend on mutability, so no caller changes. Alternatively, revert when `msg.value != 0`.

```
function mint(MintParams calldata p) external returns (
    uint256 tokenId, uint128 liquidity, uint256 amount0, uint256 amount1
) {
    if (p.recipient != msg.sender) revert BadRecipient();
    ...
}
```

SWC-131 | Unused declarations and a statement with no effect

`IERC721L` and the `V4Actions` import in `LoomLadder` are never used, `LadderMath.liquidityFor` is an external function nothing calls, `V4Pool.ticks` is never called, and eleven interfaces in `IExternal.sol` (`IV2Router`, `IFeeEscrow`, `IWNLP` and others) belong to other products. `(e);` in `_collect` only silences a warning for a parameter the function does not need.

```
// LoomLadder.sol - L8 and L15
import {IPermit2, IPositionManagerV4, IStateView, PoolKey, V4Actions, V4Pool}
    from "./interfaces/IUniswapV4.sol";
interface IERC721L {
    function transferFrom(address from, address to, uint256 tokenId) external;
}

// LadderAuto.sol - L491, inside _collect
(e);
```

Recommendation

Remove the unused interface, import and library function, and the interfaces in `IExternal.sol` that belong to other products, and drop the unused e parameter from `_collect` rather than silencing it.

```
import {IPermit2, IPositionManagerV4, IStateView, PoolKey, V4Pool}
    from "./interfaces/IUniswapV4.sol";

function _collect(
    LadderWork.Ctx memory c, LadderWork.Scratch storage s, LadderWork.Ladder storage l,
    mapping(uint256 => uint256) storage feesSince, Job memory job, address to,
    bytes memory hookData
) private returns (uint256[2] memory amt) { ... }
```

SWC-103 | Floating pragmas across three versions

The scope declares `^0.8.24` (LoomLadder, LoomZap, PilotGas, LoomLock, LadderWork, LadderAuto, LadderMath), `^0.8.26` (both adapters) and `^0.8.28` (LadderKeep). `foundry.toml` pins `0.8.28` with `via_ir`, which is what makes the deployed bytecode reproducible, but the sources alone accept other compilers, and a rebuild outside that configuration could produce different code.

```
// LoomLadder, LoomZap, PilotGas, LoomLock, LadderWork, LadderAuto, LadderMath
pragma solidity ^0.8.24;
// RamsesNpmAdapter, SlipstreamNpmAdapter
pragma solidity ^0.8.26;
// LadderKeep
pragma solidity ^0.8.28;
```

Recommendation

Pin every in-scope file to the compiler the deployment used, so the sources and `foundry.toml` cannot drift apart.

```
pragma solidity 0.8.28;
```

AUDIT COMMENTS | LoomDesk

Smart Contract audit comment for a non-technical perspective

- Owner can set the share of trading fees taken up to 5% (500 bps live), and the opening fee up to 1% of what is deposited
- Owner cannot move user funds: rescueTokens excludes position NFTs and money owed to users
- Owner can set the slip allowed on sales made for a cut up to 30% (10% live), and the pilot's sales stay within 3%
- No mint, burn, blacklist or max transaction function exists in LOOM Token



CONTRACTWOLF

Blockchain Security - Smart Contract Audits